

一些实验小提示：

- 你可以尝试调整生成的总猜测数，并探索加速比随总猜测数的变化。
- 你可以尝试调整使用的总线程数，并探索加速比随线程数的变化。
- 可供参考的论文之一：https://github.com/Ming-Xu-research/Ming-Xu-research.github.io/blob/master/_data/Parallel_PCFG_TDSC.pdf

5 NTT 选题：多线程实验

本次多线程实验分为三个方案，朴素算法，四分算法和 CRT 算法，朴素算法为基础要求，后两个为进阶要求，如果可以实现 CRT 算法，就不必再实现四分算法，但必须实现朴素算法，最终保障 CRT 算法得分不低于四分算法得分。

NTT 选题要求在本次实验不允许使用 SIMD 优化，但你可以把多线程和向量化的结合放在期末报告中。

5.1 朴素多线程优化

对于基础版的 NTT 多线程优化，实现方法较为简单，由于第二三层循环相当于遍历了一遍多项式数组，显然可以对第三层循环进行多线程优化，类似高斯消元，因此代码实现较为简单，只需要注意线程同步正确。

```
1
2 for(int mid = 1; mid < limit; mid <= 1) {
3     for(int j = 0; j < limit; j += (mid < 1)) {
4         int w = 1; // 旋转因子
5         for(int k = 0; k < mid; k++, w = w * Wn) { // 对这层循环进行优化
6             // 运算主体
7         }
8     }
9 }
```

5.2 多分多线程优化

即使在 SIMD 实验中未实现四分的 NTT，也可以在多线程这里开始。

对于非二分的 NTT, 需要判断多项式长度, 如果多项式长度不等于 4^n (即多项式长度等于 2^{2k+1}), 由于每次枚举的步长乘积等于 4, 需要预处理第一层或最后一层的循环后, 才能进行四分。

如果线程数等于 4, 每个线程负责对应需要归并的四块数据的每一块。

5.3 CRT 多线程优化

提供一个全新的多线程优化思路, 在 SIMD 的实验指导书中提到了大模数的 NTT 方法, 其本质即任意模数 NTT, 如果你是在洛谷或 oiwiki 中学习的 NTT 基础知识, 也能看到任意模数 NTT 的原理, 即使用中国剩余定理 (CRT) 合并大模数。

对于任意模数 NTT 的模板, 通常采用三模数 NTT 合并, CRT 多线程优化的思想类似任意模数 NTT, 如果让每一个线程都使用不同的小模数, 最终使用 CRT 将结果合并。

如果要实现此多线程优化, 需要在实现多模数 NTT 合并后才能实现 pthread 优化, 当模数越大时, 多线程的优化效果越明显。

你可以仿照任意模数 NTT(三模数合并) 的模板仿推出任意模数 NTT(多模数合并) 的公式, 也可以自己上网寻找相关博客, 另外以下给出了两篇应用了 CRT 合并 NTT 以加速大模数的同态加密论文, 有更详细的原理和优化的证明, 当然 CRT 合并 NTT 的重要性在超大模数 (往往远大于 64 bit) 时才明显, 对于同态加密库 (CKKS 等), 会经常使用超大模数进行加密, 因此如果实现了本优化, 要求最终需要测试一个大于 32 bit 的模数。

- A Full RNS Variant of FV like Somewhat Homomorphic Encryption Schemes
- A Full RNS Variant of Approximate Homomorphic Encryption
- Approximate CRT-Based Gadget Decomposition and Application to TFHE Blind Rotation

有关多线程优化 NTT 的论文还有很多, 可以自行搜索。

6 VTune 分析 (本地环境)

大家可以在 Intel 官网找到 VTune 的安装和使用教程。